

Gather / Design statement

WORKSHOP REGISTRATION SERVICE | NEXT.JS + SUPABASE + NETLIFY

Stack. Next.js App Router provides the browser interface and authenticated HTTP API in one TypeScript codebase. Supabase provides managed authentication, PostgreSQL transactions, and row-level security. Netlify hosts Next.js through its adapter. This keeps deployment small while putting the critical seat rule in the database. Zod validates request bodies before the API calls database functions.

Access. The permission matrix is literal: Admins manage accounts, but cannot manage or view workshops and registrations. Managers schedule workshops and handle attendees; Staff handle attendees. Each API verifies the Supabase token and reads the current profile role. RLS protects reads, and every write RPC independently checks the role. Roles never come from editable user metadata. Direct table writes are denied. Public signup is disabled in Supabase configuration and must also be disabled in the hosted project's Auth settings. The service role key is used only for server-side account creation and development seeding.

Capacity and concurrent requests. A booking transaction locks its workshop row before checking the counter. Booking, cancellation and capacity-edit transactions use the same lock. Other transactions wait, then see the updated row. Registration, counter update and audit append commit together or roll back together. An `active_count <= capacity` constraint adds protection. Cancellation is idempotent and frees a seat only once. Active duplicate email registrations are refused. This protocol is tested against the real migration in embedded PostgreSQL; a separate integration script tests simultaneous network requests against Supabase.

History. Registration records are retained forever through the UI/API, including the original booking actor/time and cancellation actor/time. Actor names are snapshotted so later profile changes do not alter historical attribution. An append-only audit table also records workshop creation/edits and account/role changes. Normal users cannot edit or delete audit rows. Operational events are visible to Managers/Staff; account events to Admins.

Experience and assumptions. A quiet catalogue gives front desk staff search, date, status, location and available-seat filters. Workshop cards show capacity and booking progress. Attendees are contact details without accounts. A location, category, duration and description supplement the spreadsheet fields. Workshop timestamps use `timestamp_tz` and render in the user's local timezone. Workshops cannot accept bookings after their start or when cancelled/completed. Managers must cancel active registrations before cancelling a workshop; reducing capacity below occupancy is refused. The last Admin cannot be demoted. Cancellation has an explicit confirmation dialog.

Trade-offs and omissions. Counter maintenance is safe because all normal writes go through these functions; service-role scripts must preserve the invariant. Account creation crosses Auth and PostgreSQL, so a failed profile provision attempts Auth cleanup. The app polls every 15 seconds instead of relying on Realtime; the final booking check still occurs inside PostgreSQL. Staff access requires real Supabase sign-in; account provisioning creates no workshop or registration data. Optional waitlisting, notification emails, advanced pagination controls, staff deactivation, password recovery and exports are omitted. Hosted Supabase tests require project access and configured credentials; a successful frontend deployment alone does not establish a configured backend.